



Knowledge-Driven Representation and Analysis of Battery Cathode Materials for Accelerated Discovery

23CSE498 – Project Phase 2 (Panel Review 1) | Team Number: 25

S.No	Register Number	Student Name
1	CB.SC.U4CSE23104	Akshay KS
2	CB.SC.U4CSE23138	R.D.Tarun
3	CB.SC.U4CSE23142	S Ajeth
4	CB.SC.U4CSE23145	Sarvesha NK
5	CB.SC.U4CSE23146	Shantharam P

Guide: **Dr. Gowtham R**

Associate Professor, Department of CSE

Co-Guide: **Dr. M. Ulaganathan**

Associate Professor, Department of Physics

September 15, 2026

Introduction

Knowledge-driven generation of stable battery cathode candidates

- Discovering viable battery cathode materials by trial and error is slow – most candidates fail on structural or chemical grounds that domain knowledge could have ruled out early.
- **Our approach:** encode that domain knowledge as an ontology, embed it as a continuous conditioning signal, and use it to steer a pretrained crystal-generation model toward candidates that respect it.
- Generated formulas are carried through to real 3D structures and screened down to DFT-validated, experimentally testable candidates.
- The rest of this talk walks through each stage: the knowledge layer, the embedding, the conditioning mechanism, and the generation-to-validation pipeline.

Problem Statement

What the knowledge-driven discovery pipeline needs to solve

To accelerate discovery of viable battery cathode materials, the pipeline must:

- Unify heterogeneous structural, computational, and experimental data into a single, queryable knowledge base.
- Turn that knowledge into a continuous signal a generative model can actually condition on, not just a static schema.
- Steer crystal generation toward structurally and chemically valid candidates, instead of searching blindly.
- Carry generated formulas through to real 3D structures and screen them for stability and synthesizability.
- Close the loop with DFT validation and, where possible, experimental confirmation.

Application Architecture & Workflow Design



Domain knowledge is embedded and used to condition crystal generation; the resulting formulas are carried through to real 3D structures and screened down to validated candidates – each stage is detailed in the sections that follow.

 knowledge & conditioning path  downstream generation & validation

Implementation Progress: Module-Level Status

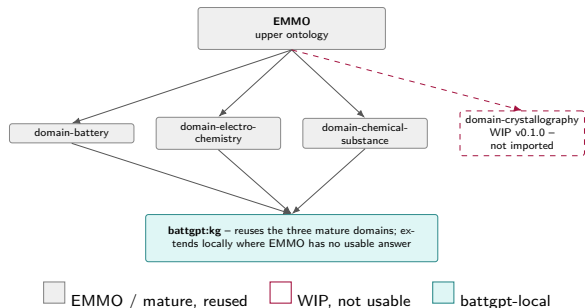
Component-by-component state of the knowledge-conditioned pipeline

Module / Component	Current Implementation State	Status
Ontology (battgpt:kg)	v0.3.2 – 894 triples, 32 classes, 28 object / 30 datatype properties; two local extensions; cathode instances populated	Functional
OnT hyperbolic embedding	Poincaré ball embeddings over the merged ontology closure; centripetal radial ordering verified	Functional
Ontology → pipeline bridge	Deterministic ancestor traversal, memory-bank construction, persisted run manifest	Functional
Conditioning layer	Zero-initialised gated cross-attention wired into frozen CrystaLLM blocks	Integrated
Structure-family classifier	pymatgen space-group + composition rules across all nine leaf families	Functional
Generation & validation	End-to-end CIF generation with an independently grounded validation gate	In Progress
MatterGen handoff	Formula extraction to seed the team's MatterGen track	Planned

What is EMMO?

Scope, current extensions, and the case for extending it further

- **EMMO** [1]: a foundational, axiomatic upper ontology for materials science, grounded in physics and analytical philosophy.
- **Mature domains reused as-is:** domain-battery (BattINFO-aligned [2]), domain-electrochemistry, domain-chemical-substance.
- **domain-crystallography is not usable:** an explicit work-in-progress v0.1.0 draft – confirmed by direct inspection, not imported by anyone.
- **Why we extend:** generation needs machine-readable crystallographic structure aligned with DFT output, and a structure-family classification vocabulary – neither exists upstream.
- Battery chemistry is *already* covered



Competency Questions

What the ontology must be able to answer

Crystallographic structure

- CQ1. What are the unit cell parameters and space group of a given crystal structure?
- CQ2. Which sites and species occupy the structure, at what fractional coordinates, and in what coordination geometry?
- CQ3. Which bonds exist between which sites, at what distance, and by what determination method?
- CQ4. What DFT-calculated properties (band gap, formation energy, energy above hull, bulk/shear modulus) does a material have, under which computational method?

Structural classification

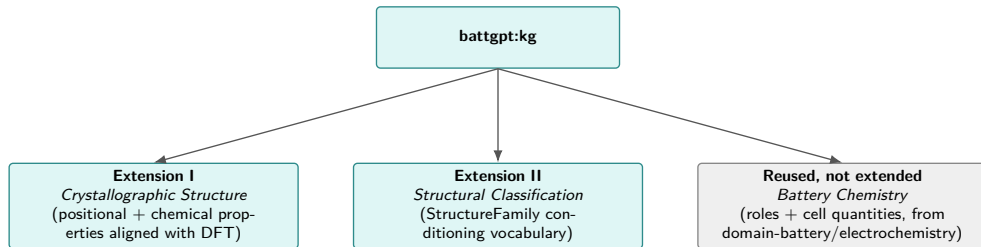
- CQ5. Which structural-prototype family (layered oxide, olivine, spinel, garnet, NASICON, sulfide superionic, ...) does a structure belong to?
- CQ6. Is a material's family classification unique and reasoner-checkable, not just convention?

Battery chemistry (reused, not extended)

- CQ7. What functional role (positive electrode, negative electrode, electrolyte, separator) does a material play in a cell?
- CQ8. What are a cell's rated electrochemical properties (capacity, voltage, ionic conductivity, ...)?

Ontology Schema Overview

Two local extensions, one domain reused as-is

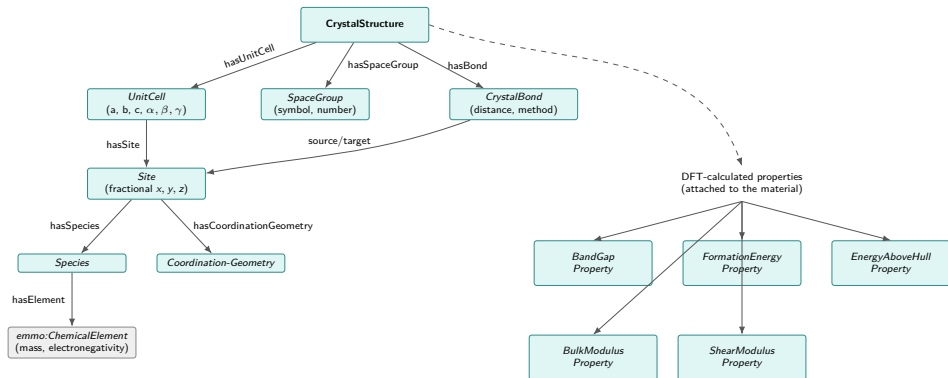


 defined locally by this project

 imported from EMMO, used as-is

Extension I: Crystallographic Structure

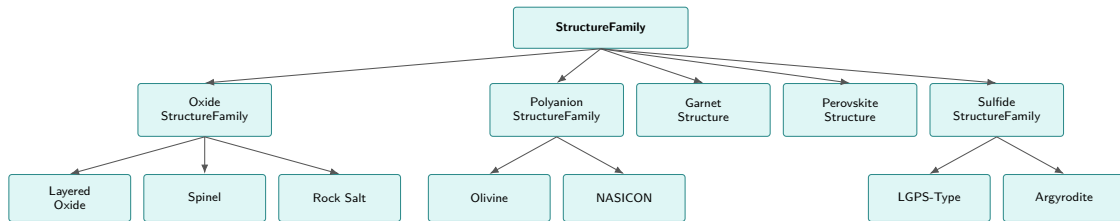
Positional & chemical properties aligned with DFT-calculated properties



Justification: We ground **CrystalStructure** directly under **owl:Thing** because EMMO's domain-crystallography is still an unreleased v0.1.0 draft that nothing else imports. This is the only place we represent positional and chemical structure in a form that lines up with what DFT actually computes.

Extension II: Structural Classification

The StructureFamily conditioning vocabulary



Each leaf is anchored to a canonical example from our pipeline's curated Materials Project records [3] – $\text{LiCoO}_2/\text{LiNiO}_2$, $\text{LiMn}_2\text{O}_4/\text{Li}_4\text{Ti}_5\text{O}_{12}$, $\text{LiFePO}_4/\text{NaFePO}_4$, $\text{Na}_3\text{V}_2(\text{PO}_4)_3$, LGPS, LLZO. All nine leaves are owl:AllDisjointClasses

Justification: We checked every relevant EMMO domain – the ones we import (chemical-substance, battery, electrochemistry) and the not-yet-usable domain-crystallography draft – and none of them define this structural-prototype vocabulary, so we built it locally. This is the conditioning vocabulary CrystaLLM and OnT need; deliberately the deepest subclass hierarchy in the ontology.

Scope & Limitations

Where the schema currently stands

- **Instances are populated, but only a cathode slice:** real materials (e.g. LiFePO_4) are populated end-to-end, not yet spanning every StructureFamily leaf or the electrolyte/separator roles.
- **Few places still need formal constraints:** e.g. DFT properties already carry a value and a unit in practice, but the ontology doesn't yet require it – this is one of several such gaps, and tightening them is an ongoing refinement.
- **Provenance is tracked, reconciliation is not:** each property and bond records its source method, but comparing the same quantity across multiple databases for one material is still open.
- **Consistency is now reasoner-checked:** every ontology change is validated with ELK [4] (EL profile) and HermiT (full consistency) before it's applied – zero new unsatisfiable classes introduced so far.

The Core Question: How Does a Model Understand an Ontology?

Moving from human-readable symbolic graphs to machine-executable representations

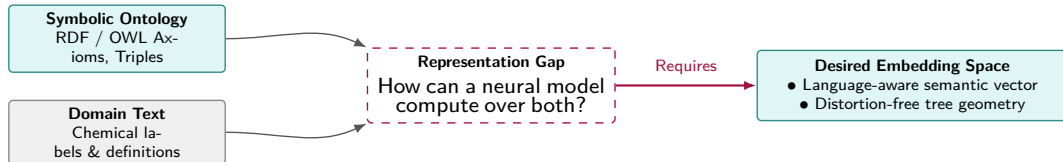
1. It Must Understand English Semantics

- Classes carry rich descriptive domain text (*"delithiated high-nickel layered oxide"*).
- Relations encode physical logic (*hasCoordinationGeometry*) requiring textual grounding.
- **Failure of pure IDs:** Discrete graph tokens discard chemical and linguistic meanings.

2. It Must Preserve Strict Hierarchies

- Subsumption is asymmetric: every *Layered Oxide* is an *Oxide*, but not vice-versa.
- Distance must reflect taxonomic depth and shared parentage rather than surface word similarity.
- **Failure of standard PLMs:** Flat Euclidean space causes metric distortion and tree crowding.

The Challenge: How do we map symbolic logic into a continuous space that preserves **both natural-language semantics** and **hierarchical tree geometry**?



OnT: Hyperbolic Ontology Embeddings

Unifying language models and axiomatic hierarchies in non-Euclidean space

What is OnT?

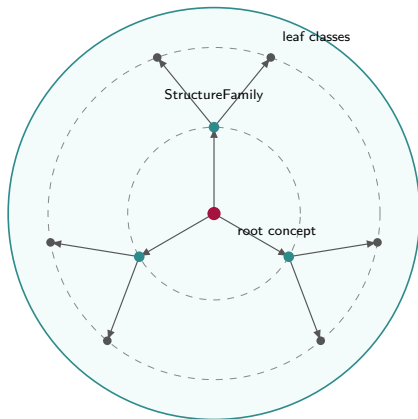
- Tunes a **PLM** via geometric modeling in hyperbolic space.
- Preserves Description Logic $\mathcal{EL}$ axioms ($C \sqsubseteq D$, $\exists r.C$, $C \sqcap D$) and text.

How It Works

- **Verbalizer + PLM:** Converts complex concepts into text ($\mathcal{V}(C)$).
- **Roles as Rotations:** Models roles via rotation $R(\Theta_r)$ and scale k_r .
- **Hierarchy Losses:** Optimizes via *Contrastive* and *Centripetal* losses.

Why Hyperbolic Space?

- **Exponential Volume:** Radius growth matches tree branching rates.
- **Zero Leaf Crowding:** Prevents metric collapse between deep leaves.



Poincaré ball – depth is radius

Why a Knowledge Graph, Not Only the Ontology

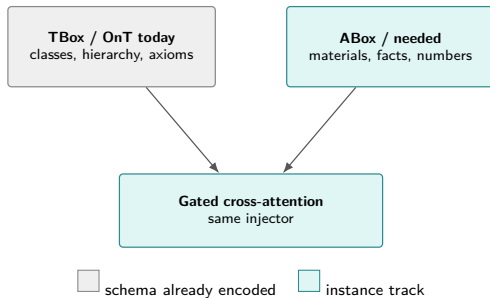
Same pipeline, two objects: schema vs instances

The missing instance information

- **OnT encodes the TBox:** concepts, hierarchy, and ontology relations [?].
- A **class embedding is shared:** LiFePO_4 and NMC811 may both be instances of Substance, but their properties differ.
- The populated KG provides the **ABox:** materials, crystals, properties, and relations.

Parallel, not a replacement

- **TBox / Ontology:** what kinds of things exist and how they are related.
- **ABox / KG:** which specific things exist and what is true about them.
- Encode both and provide the **material-specific representations** to the cross-attention injector in **CrystaLLM** [?].



Existing Encoders: Strengths and Gaps

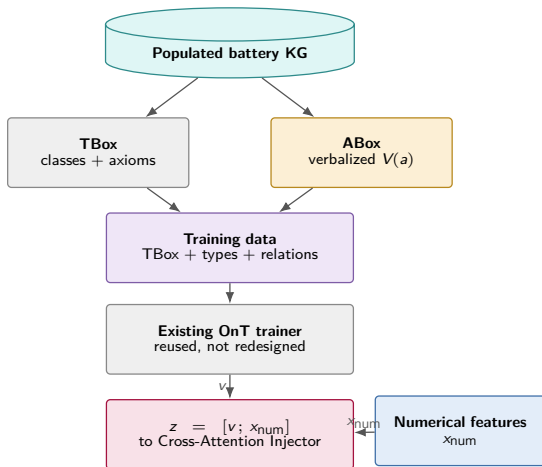
No single method combines all signals required by our KG

Method	What it captures	Gap for our setting
BoxEL [5]	Classes as boxes, individuals as points, and relations as transformations. TBox + ABox logical constraints.	No native scientific text or dedicated numerical representation.
Box²EL [6]	Improved EL++ representation with concept boxes, individual points, and richer role representation. TBox + ABox .	No language-model or dedicated concrete-value encoder.
SimKGC [7]	Language-model representations of entity/relation text for KG link prediction.	No ontology-specific semantics or dedicated numerical representation.
KEPLER [8]	Language-model entity descriptions jointly trained with TransE for KG representation.	Not designed for ontology-specific semantics or scientific numerical features.
RotatE [9]	Learned entity/relation embeddings for KG structure and link prediction.	Does not directly exploit our node types or numerical features.

Our requirement: TBox semantics + ABox instances + scientific attributes → **material embeddings** for CrystaLLM [?] (OnT [?] is the encoder we extend)

Planned Architecture: Extending OnT to the Populated KG

Verbalize instances, reuse OnT, add a numerical feature channel



What we do

- Keep OnT for the **TBox**. Verbalize each material and encode it with the **same PLM/OnT pipeline** – one encoder, not a second model.
- Extend OnT's hierarchy and role losses to ABox types and relations, and evaluate the resulting instance geometry.
- After OnT, attach scaled numbers: v is the hyperbolic embedding; $z = [v; x_{num}]$ is what CrystaLLM sees.
- Retrieve **material-specific** z into the existing Cross-Attention Injector. CrystaLLM stays frozen.

The Need for a Generative Model?

The gap between a formula and a usable 3D crystal structure

The Problem

- Our ontology-conditioned pipeline outputs a **chemical formula** – not a 3D structure.
- Existing databases (Materials Project, ICSD) cover known materials only – novel cathode candidates are by definition absent.
- Brute-force DFT enumeration of all possible structures for a formula is computationally infeasible.

What the Model Must Do

- Given a formula (and optionally a target symmetry), **propose valid 3D crystal structures** on demand.
- Respect crystallographic constraints: space group, Wyckoff sites, unit cell geometry.
- Accept external conditioning signals – our ontology knowledge must actually influence what gets generated.

The question then becomes: what *type* of generative model satisfies all three requirements?

The Core Question: Auto-Regressive Generative Model

From disordered diffusion to exact, conditioned structure generation

1. Crystal Generation Must Preserve Symmetry

- A valid crystal is governed by a space group – every atomic position is constrained to Wyckoff sites.
- Continuous diffusion drifts across 1,000 denoising steps, routinely collapsing into disordered $P1$.
- **CrystaLLM emits the space group first:** all coordinates are conditioned on it – symmetry by construction.

2. Generation Must Accept External Signals

- Battery discovery requires steering toward target compositions and structural families – not random sampling.
- Score-function conditioning in diffusion models is incompatible with discrete ontology signals.
- **CrystaLLM accepts a prefix:** formula, space group, or atomic properties – exactly what our ontology provides.

Key Insight: CIF text sequences enforce symmetry exactly and accept ontology prefixes naturally – the right fit for knowledge-conditioned discovery.



CrystaLLM: Overview [?]

Autoregressive crystal generation via flexible prefix prompting

What is CrystaLLM?

- **Crystals as Text:** A decoder-only Transformer (GPT) that generates periodic 3D crystals as discrete CIF tokens.
- **Prefix Autocompletion:** Operates like code completion: users provide a partial CIF prefix prompt, and the model autocompletes the remaining structure.

Why We Use It for Battery Discovery:

- **Exact Symmetry Guarantee:** Lock in target space groups ($R\bar{3}m$, $pnma$); coordinates are strictly bound to symmetry sites.
- **High Throughput:** Generates candidate structures in seconds on a single GPU.
- **Flexible Guidance:** Allows chemical and physical steering via prompt prefixing.

Two Ways of Input ($\text{Li}_3\text{Mn}_2\text{NiO}_6$)

Mode 1: Minimal Formula Prompt

```
data_Li3Mn2NiO6
```

→ Unconstrained exploration: predicts space group, cell dimensions, and atomic coordinates from scratch.

Mode 2: Rich Symmetry Prompt

```
data_Li3Mn2NiO6
```

```
loop_
```

```
  _atom_type_symbol _electronegativity _radius
```

```
Li    0.9800    1.4500
```

```
Mn    1.5500    1.4000
```

```
Ni    1.9100    1.3500
```

```
O     3.4400    0.6000
```

```
_symmetry_space_group_name_H-M R-3m
```

→ Locks in layered cathode symmetry; model autocompletes cell dimensions and atomic sites.

CrystaLLM Working: Sequential Generation Pipeline [?]

From a partial conditioning prompt to a complete CIF file



Token-by-Token Autoregression:

- Predicts the next token based on all preceding context:

$$P(\mathbf{x}) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}; \theta)$$

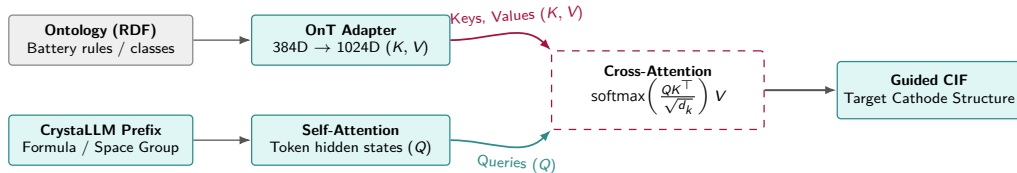
- Emits one character or coordinate at a time until the entire 3D crystal file is complete.

The Vanilla Baseline Mechanism:

- **Causal Self-Attention:** Computes attention strictly between past text tokens to learn local bond lengths and angles.
- **The Core Limitation:** Over long CIF sequences (> 500 tokens for 20+ atoms), attention attenuates and forgets initial constraints.
- **Motivation for Modification:** Needs persistent external guidance throughout the sequence to maintain target battery properties.

Our Modifications to CrystaLLM: Cross-Attention Injection [?]

Steering crystal generation with ontology knowledge at every transformer layer



Attention operates as a **soft dictionary lookup**:

- **Query Q:** What the current crystal token is looking for (from CrystaLLM hidden states).
- **Key K:** What each ontology concept advertises about itself (from OnT embeddings).
- **Value V:** What structural constraints the ontology hands over to steer generation.

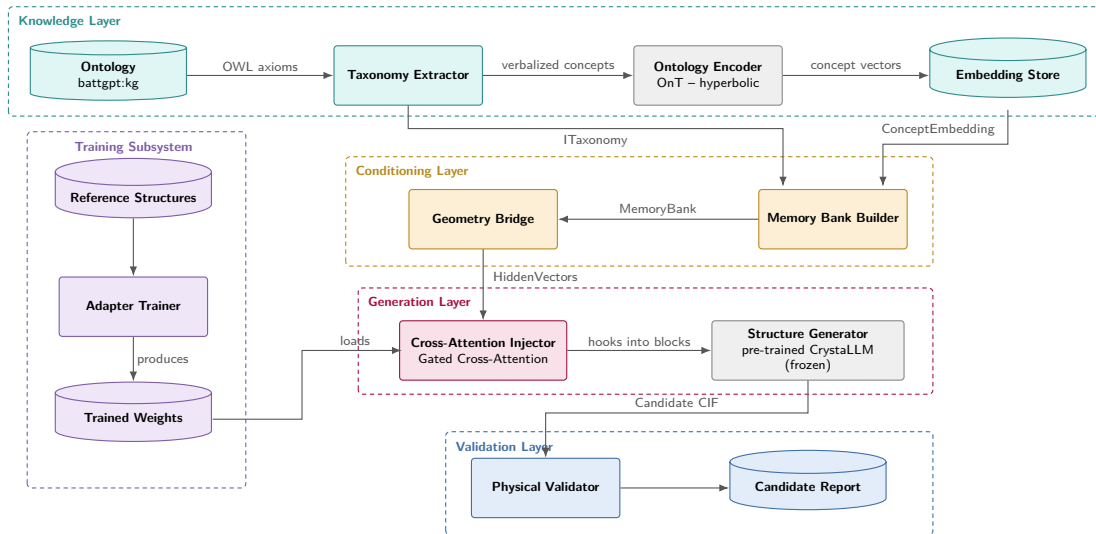
Mathematical Formulation:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- **Continuous Steering:** Injected into every transformer layer so CrystaLLM references cathode rules at each token step.

System Architecture

End-to-end component view: knowledge $\rightarrow$ conditioning $\rightarrow$ generation $\rightarrow$ validation



From Formula Generation to Crystal Discovery

Oxidation-State Enumeration as the Bridge to Structure Generation

Formula obtained from Knowledge-Conditioned CrystaLLM



Input to the Materials Pipeline

- **Chemical formula** obtained from CrystaLLM.
- The formula defines the target **elemental composition**.
- The composition is used to enumerate **chemically feasible oxidation-state configurations**.

Why Oxidation-State Enumeration?

- A composition can admit **multiple oxidation-state configurations**.
- Each valid configuration represents a **distinct electronic constraint**.
- Each configuration becomes a **conditioning input** for crystal generation.

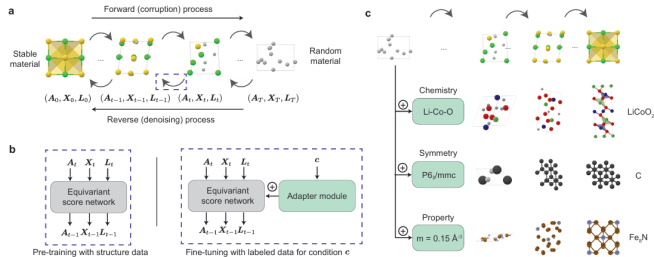
MatterGen Adapter Fine-Tuning

Leveraging Parameter-Efficient Conditioning of the Pretrained Generator

Why Fine-Tune MatterGen?

- MatterGen is a pretrained diffusion model for inorganic crystal structure generation.
- The pretrained model does not explicitly condition generation on **oxidation states**.
- MatterGen provides an existing **adapter-based fine-tuning architecture**.
- We leverage this architecture to introduce **oxidation-state conditioning**.

General Adapter Mechanism



General adapter-based conditioning mechanism used by the pretrained architecture. [?]

Key Idea

Adapt the pretrained generator
without retraining the complete model.

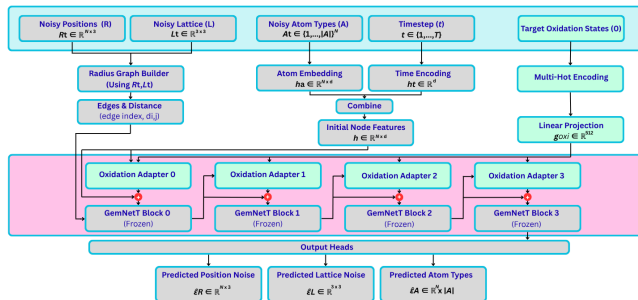
Oxidation-State Conditioning Architecture

Adapting MatterGen to Generate Structures Under Oxidation-State Conditions

Oxidation-State Conditioning

- 1 Oxidation-State Encoding**
Each configuration is represented as a categorical vector.
- 2 Embedding**
The encoded configuration is mapped to a dense oxidation-state representation.
- 3 Adapter MLP**
Learns how oxidation-state information influences the pretrained representation.
- 4 Zero-Initialized Mix-in**
Injects the adapter output into the GemNet message-passing layers.

Adapter Integration



Parameter-Efficient Training

Backbone: Frozen | Adapters: Trainable

Fine-Tuning Dataset & Evaluation

Oxidation-State Labelling and Generation Consistency

Dataset Construction

- 1 **MP20**
Lithium-based transition-metal oxide structures were extracted.
- 2 **Oxidation-State Labelling**
Oxidation states were assigned using **bond-valence analysis[?]**.
- 3 **Quality Filtering**
Structures with **fractional or ambiguous** oxidation states were discarded.
- 4 **Final Dataset**
Remaining structures were used for oxidation-state-conditioned fine-tuning.

Training Dataset

MP20 → Li-O structures → oxidation-state labelling → quality filtering

Generation & Evaluation

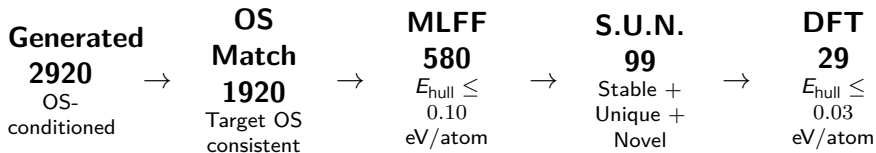
- 1000 structures generated for **Li-TM-O**.
- Composition filtering: **120 formulas**.
- Oxidation-state enumeration[?]: **146 pairs**.
- **20 structures/pair** → **2920 structures**.
- **1000** oxidation-state mismatches.
- **1920** structures matched target oxidation states.

Oxidation-State Consistency

1920 / 2920 = 65.75%
structures matched the target configuration

Hierarchical Screening & Candidate Selection

Progressive Filtering from Generated Structures to Experimental Validation



1. MLFF Screening

- Relaxation using **MatterSim MLFF**. [?]
- Stability evaluated using energy above convex hull.
- **580** structures: $E_{\text{hull}} \leq 0.10$ eV/atom.

2. S.U.N. Screening [?]

- **Stable**: energetically favourable structures.
- **Unique**: remove duplicate polymorphs.
- **Novel**: remove previously reported structures.
- **580** → **99**

3. DFT Validation[?]

- Full DFT relaxation and stability evaluation.
- **29** structures: $E_{\text{hull}} \leq 0.03$ eV/atom.
- Electrochemical analysis performed on the 29 candidates.

Selected: $\text{Li}_3\text{Mn}_2\text{NiO}_6$ | 281.74 mAh/g | 3.451 V | $E_{\text{hull}} = 0.0227$ eV/atom

Experimental Validation: $\text{Li}_3\text{Mn}_2\text{NiO}_6$ was synthesized and characterized by XRD.

Next Milestones

Closing the ontology loop, and proving the conditioning works

Ontology track

- 1 **Broaden instance coverage** beyond the current cathode slice, populating every battery role and structural family, not just cathodes.
- 2 **Validate against every competency question** by running each one as a real query against the populated graph, closing the engineering loop end-to-end.
- 3 **Keep enriching structural semantics** so structure-family membership carries real geometric and crystallographic meaning, not just a taxonomic label.
- 4 **Score the ontology against OQuaRE** to benchmark its structural and design quality against established ontology-evaluation metrics.

Conditioning pipeline

- 1 **OnT – extend to the populated KG.** The encoder currently embeds the **TBox**; extend it toward **ABox instance embeddings** by verbalizing material entities and reusing its hierarchy and role mechanisms.
- 2 **Generation – widen the target set.** Run conditioned CrystaLLM for at least one generation target per StructureFamily leaf.
- 3 **Output – demonstrate ontology-driven generation.** Show generated CIFs landing in the requested family, turning “the ontology is wired in” into “the ontology changes what comes out.”
- 4 **Seed the MatterGen track** with formulas extracted from conditioned output, completing the planned merger.

Technical Enrichment Activity Choices

Professional Online Certification Details (Individual Mandatory: 10–20 Hours)

● Individual MOOC / Online Certification Selection:

S.No	Student Name	Chosen MOOC / Course Title	Platform	Hours
1	Akshay KS	Knowledge Graphs – Foundations and Applications	openHPI	15 Hrs
2	R.D.Tarun	Knowledge Graphs – Foundations and Applications	openHPI	15 Hrs
3	S Ajeth	Knowledge Graphs – Foundations and Applications	openHPI	15 Hrs
4	Sarvesha NK	Knowledge Graphs – Foundations and Applications	openHPI	15 Hrs
5	Shantharam P	Knowledge Graphs – Foundations and Applications	openHPI	15 Hrs

- **Course:** <https://open.hpi.de/courses/knowledgegraphs2023> – all five team members have registered for this course together, directly relevant to the ontology and knowledge-graph track of this project.

References I

-  P. Del Nostro, J. Friis, E. Ghedini, G. Goldbeck, O. Holtz, O. Roscioni, F. A. Zaccarini, and D. Toti, “Elementary multiperspective material ontology: Leveraging perspectives via a showcase of EMMO-based domain and application ontologies,” in *Proceedings of the 16th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management (KEOD)*, 2024, pp. 135–142.
-  S. Clark, F. L. Bleken, S. Stier, E. Flores, C. W. Andersen, M. Marcinek, A. Szczesna-Chrzan, M. Gaberscšek, R. Palacin, M. Uhrin, and J. Friis, “Toward a unified description of battery data,” *Advanced Energy Materials*, vol. 12, no. 17, p. 2102702, 2022.
-  A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, and K. A. Persson, “Commentary: The Materials Project: A materials genome approach to accelerating materials innovation,” *APL Materials*, vol. 1, no. 1, p. 011002, 2013.

References II

-  Y. Kazakov, M. Krötzsch, and F. Simaňík, “The incredible ELK: From polynomial procedures to efficient reasoning with EL ontologies,” *Journal of Automated Reasoning*, vol. 53, no. 1, pp. 1–61, 2014.
-  B. Xiong, N. Potyka, T.-K. Tran, M. Nayyeri, and S. Staab, “Faithful embeddings for EL++ knowledge bases,” in *The Semantic Web – ISWC 2022*. Springer, 2022, pp. 22–38.
-  M. Jackermeier, J. Chen, and I. Horrocks, “Dual box embeddings for the description logic EL++,” in *Proceedings of the ACM Web Conference 2024*, ser. WWW '24, 2024.
-  L. Wang, W. Zhao, Z. Wei, and J. Liu, “SimKGC: Simple contrastive knowledge graph completion with pre-trained language models,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2022, pp. 4281–4294.

References III

-  X. Wang, T. Gao, Z. Zhu, Z. Zhang, Z. Liu, J. Li, and J. Tang, “KEPLER: A unified model for knowledge embedding and pre-trained language representation,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 176–194, 2021.
-  Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang, “RotatE: Knowledge graph embedding by relational rotation in complex space,” in *International Conference on Learning Representations*, 2019.
-  Z. Hu, Y. Dong, K. Wang, and Y. Sun, “Heterogeneous graph transformer,” in *Proceedings of The Web Conference 2020*, ser. WWW '20, 2020, pp. 2704–2710.
-  X. Tang, S.-C. Zhu, Y. Liang, and M. Zhang, “RuleE: Knowledge graph reasoning with rule embedding,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

Thank You